Registre des dettes techniques — FileFlow

Titre	Registre des dettes techniques — Plateforme FileFlow
Version	1.3
Date	27 octobre 2026
Statut	Validé
Propriétaire	Driss OUAAZIZ — Software Architect
Classification	Interne

Table des matières

1. Objet
2. Définition et cadran de Fowler
3. Processus de gestion
4. Registre
5. Détail des entrées
6. Plan de résorption priorisé
7. Références

1. Objet

Ce registre recense, qualifie et priorise les dettes techniques identifiées sur la plateforme **FileFlow**. Il constitue l'outil de pilotage de leur résorption et alimente la planification des travaux de fiabilisation. Il est tenu à jour à chaque revue d'architecture et à chaque évolution majeure.

Le terme « dette technique » désigne ici tout écart, compromis ou report d'implémentation qui facilite une livraison à court terme au prix d'un coût de maintenance, de risque ou d'effort accru à moyen terme.

2. Définition et cadran de Fowler

La notion de dette technique, introduite par **Ward Cunningham** (1992), est qualifiée selon le **cadran de Martin Fowler**, qui croise l'intention (délibérée / par imprudence) et la lucidité (prudente / imprudente) :

	PRUDENTE	IMPRUDENTE
Délibérée	« Nous livrons maintenant et en assumons les conséquences »	« Nous n'avons pas le temps de bien faire »
Par imprudence	« À présent nous savons comment nous aurions dû faire »	« Qu'est-ce que la conception en couches ? »

Chaque entrée du registre est classée dans l'un de ces quatre quadrants, afin de distinguer les compromis assumés des écarts à corriger en priorité.

3. Processus de gestion

1. **Identification** : toute dette détectée (revue de code, audit, incident, scan de sécurité) est consignée dans le registre avec un identifiant unique.
2. **Qualification** : catégorie, quadrant de Fowler, impact, effort estimé.
3. **Priorisation** : combinaison impact × probabilité × effort ; priorité de P1 (élevée) à P3 (faible).
4. **Suivi** : statut *Ouverte*, *Planifiée*, *En cours*, *Acceptée* (risque assumé et documenté), *Résolue*.
5. **Revue** : passage en revue à chaque itération de fiabilisation et à chaque montée de version majeure.

Échelle d'effort indicative : **S** (≤ 1 j), **M** (2–5 j), **L** (> 5 j).

---

4. Registre

ID	DESCRIPTION	CATÉGORIE	QUADRANT FOWLER	IMPACT	EFFORT	PRIORITÉ	STATUT
D1	Métadonnées/tags non « rebasés » lors d'un rename/move	Correction / cohérence	Délibérée prudente	Moyen	M	**P1**	Résolue
D2	Clés i18n orphelines après retrait de la prévisualisation PowerPoint	Propreté du code	Par imprudence prudente	Faible	S	P3	Résolue
D3	Image statique non-PIE (perte d'ASLR)	Durcissement sécurité	Délibérée prudente	Moyen	M	P2	Acceptée
D4	Avis `rsa` RUSTSEC-2023-0071 et `crypto-bigint` RUSTSEC-2026-0387 (chaîne transitive)	Dépendance / sécurité	Délibérée prudente	Faible	L	P3	Acceptée
D5	Couverture DAST partielle (baseline non authentifiée)	Assurance sécurité	Délibérée prudente	Moyen	M	**P1**	Planifiée
D6	`package-lock.json` vestigial maintenu en parallèle de pnpm	Cohérence de build	Par imprudence prudente	Faible	S	P2	Résolue
D7	Absence de tests frontend automatisés	Qualité / tests	Délibérée imprudente	Moyen	L	**P1**	Résolue
D8	Connexion SQLite unique sous verrou (Mutex)	Scalabilité / performance	Délibérée prudente	Moyen	L	P2	Ouverte
D9	En-têtes `Permissions-Policy` / COOP-COEP non positionnés	Durcissement sécurité	Par imprudence prudente	Faible	S	P2	Résolue
D10	Image de build front-end `node:20-slim` (Node 20 hors support) — N2	Dépendance / sécurité	Par imprudence prudente	Faible	S	P2	Ouverte
D11	Images Docker de base non épinglées par empreinte — N3	Chaîne d'approvisionnement	Délibérée prudente	Faible	S	P2	Ouverte

ID	DESCRIPTION	CATÉGORIE	QUADRANT FOWLER	IMPACT	EFFORT	PRIORITÉ	STATUT
D12	Chaîne d'outils Rust non figée en CI — N4	Cohérence de build	Délibérée prudente	Faible	S	P2	Ouverte
D13	Montées de version majeures proposées par Dependabot non instruites	Dépendances	Délibérée prudente	Moyen	L	P2	Ouverte
D14	Règles ESLint d'hygiène React maintenues en avertissement	Qualité du code	Délibérée prudente	Faible	M	P3	Ouverte
D15	Vérifications de sécurité non converties en tests d'intégration exécutés en CI	Qualité / tests	Délibérée prudente	Moyen	L	**P1**	Planifiée
D16	Verrouillage d'un lien public protégé par un tiers — N1	Sécurité / disponibilité	Délibérée prudente	Faible	S	P2	Ouverte
D17	Lien public créable par le destinataire d'un partage interne — N5	Contrôle d'accès	Par imprudence prudente	Moyen	S	**P1**	Ouverte
D18	Compteurs de verrouillage anti-force-brute tenus en mémoire	Robustesse / sécurité	Délibérée prudente	Faible	M	P3	Ouverte

5. Détail des entrées

D1 — Métadonnées/tags non rebasés lors d'un rename/move

Contexte. Lorsqu'un fichier ou dossier est renommé ou déplacé, les partages et les favoris sont « rebasés » (leur chemin de référence est mis à jour). Les **métadonnées/tags** (associés au chemin dans `metadata.json`) ne le sont pas. **Risque.** Un tag reste rattaché à l'ancien chemin ; si un nouveau fichier est créé ultérieurement au même chemin, il hérite indûment des tags de l'ancien. Impact fonctionnel (cohérence des métadonnées), non critique pour la sécurité d'accès. **Remédiation proposée.** Étendre la logique de rebasage déjà appliquée aux partages et favoris aux entrées de `metadata.json` lors des opérations de renommage et de déplacement, avec test de non-régression. **Résolution (v1.1).** Les étiquettes suivent désormais le renommage et le déplacement (chemin et descendants) et sont purgées à la mise en corbeille, comme les partages et les favoris. Tests de non-régression ajoutés.

D2 — Clés i18n orphelines

Contexte. Le retrait de la prévisualisation PowerPoint a laissé des clés de traduction inutilisées dans les fichiers de langue. **Risque.** Aucun impact fonctionnel ; bruit et dérive documentaire des fichiers d'internationalisation. **Remédiation proposée.** Supprimer les clés orphelines et mettre en place une vérification de clés inutilisées dans la chaîne de qualité. **Résolution (v1.1).** 29 clés orphelines supprimées (prévisualisation PowerPoint et reliquats de l'ancienne navigation). Le script `pnpm run check:i18n` vérifie la parité EN/FR et l'absence de clés non référencées.

D3 — Image statique non-PIE (perte d'ASLR)

Contexte. L'obtention d'un binaire entièrement statique exécutable sur `distroless/static` a nécessité une compilation **non-PIE** (`relocation-model=static`), ce qui désactive la randomisation d'espace d'adressage

(ASLR) du binaire principal. **Risque.** Réduction d'une mesure de durcissement contre l'exploitation mémoire. Le risque résiduel est faible pour une application écrite en Rust (sûreté mémoire par construction, usage minimal de code *unsafe*). **Décision.** Compromis **accepté** : bénéfice majeur (image de 11,3 Mo, zéro CVE de base) supérieur au risque résiduel. À réévaluer si une alternative statique compatible PIE devient disponible.

D4 — Avis `rsa` RUSTSEC-2023-0071

Contexte. Le crate `rsa` (vulnérabilité de canal auxiliaire temporel « Marvin ») est tiré de manière transitive par la bibliothèque de jetons pour la prise en charge de RS256. FileFlow n'émet et ne vérifie que des jetons **HS256** et n'exécute donc **aucune opération RSA**. **Risque.** Le chemin vulnérable est **inatteignable** ; aucun correctif amont n'est disponible. **Décision.** Compromis **accepté** et documenté dans la configuration d'audit (`.cargo/audit.toml`). L'exception sera levée si des jetons signés RSA sont un jour introduits. **Mise à jour (v1.3).** Un second avis de même nature, RUSTSEC-2026-0387 (`crypto-bigint` 0.5, défauts de temps constant), est tiré par la même bibliothèque pour ES256/ES384. Les jetons étant exclusivement HS256 (clés HMAC), aucune opération sur courbe elliptique n'est exécutée : exception justifiée ajoutée dans `backend/.cargo/audit.toml` et dans `backend/deny.toml` (désormais contrôlé en CI par `cargo-deny`). Les deux exceptions seront levées dès qu'une version de la bibliothèque de jetons ne tirera plus ces crates ; la vérification est faite à chaque montée de version proposée par Dependabot.

D5 — Couverture DAST partielle

Contexte. Le test dynamique de sécurité réalisé est une **analyse de référence non authentifiée** : le parcours n'a couvert que la surface publique (interface, ressources statiques, `/api/health`). **Risque.** La surface authentifiée de l'API (fichiers, partages, téléversements, administration, double authentification) n'a pas encore été testée dynamiquement. **Remédiation proposée.** Réaliser un **scan actif authentifié** de l'API, pilotable par un contrat d'interface (OpenAPI) et une session administrateur, en excluant les points de terminaison de déconnexion. À intégrer au cycle de tests de sécurité.

D6 — `package-lock.json` vestigial

Contexte. La construction officielle repose sur **pnpm** (`pnpm-lock.yaml`, installation à verrou figé). Un `package-lock.json` subsiste, maintenu par cohérence lors des mises à jour de dépendances. **Risque.** Double source de vérité potentielle sur les versions de dépendances ; risque de divergence. **Remédiation proposée.** Décider d'une source unique : soit supprimer `package-lock.json`, soit formaliser sa synchronisation dans le processus de mise à jour. **Résolution (v1.1).** `frontend/package-lock.json` supprimé et ignoré ; le `Makefile` utilise pnpm (même version que l'image), `pnpm-lock.yaml` est l'unique source de vérité.

D7 — Absence de tests frontend automatisés

Contexte. Le frontend ne dispose que d'une vérification de types et d'un contrôle de build ; il n'existe pas de tests unitaires ni d'intégration automatisés. **Risque.** Régressions fonctionnelles non détectées côté interface, notamment sur les composants sensibles (rendu de contenu, gestion des sessions, flux de partage). **Remédiation proposée.** Introduire une base de tests unitaires et de composants sur les modules critiques, puis l'intégrer à la chaîne d'intégration continue. **Résolution (v1.2).** Socle Vitest + Testing Library (104 tests) couvrant l'assainissement HTML (charges XSS), le rendu Markdown, le client d'API (gestion d'erreurs, téléversement reprenable), le store de navigation, le parcours de connexion avec double authentification et la création de liens de partage. Exécuté par `pnpm test` et par le nouveau workflow `ci.yml` (tests, parité i18n, vérification de types et build frontend ; tests backend). Ces tests ont mis au jour et permis de corriger deux anomalies : le double échappement de `&` dans les URL du rendu Markdown et la saisie du nombre maximal de téléchargements, qui ne pouvait pas être effacée.

D8 — Connexion SQLite unique sous verrou

Contexte. L'accès à la base s'appuie sur une **connexion unique protégée par un verrou (Mutex)**, ce qui sérialise les écritures. Ce choix est suffisant à l'échelle actuelle. **Risque.** En cas de forte concurrence en écriture, ce verrou peut devenir un point de contention (impact sur la latence — voir alerte A3, document 11). **Remédiation proposée.** Évaluer l'introduction d'un pool de connexions (par ex. r2d2) si la charge d'écriture le justifie, sous réserve de la compatibilité avec le modèle WAL.

D9 — En-têtes `Permissions-Policy` / COOP-COEP

Contexte. L'analyse dynamique a signalé l'absence des en-têtes `Permissions-Policy` et, optionnellement, `Cross-Origin-Opener-Policy` / `Cross-Origin-Embedder-Policy`. **Risque.** Durcissement HTTP incomplet ; sévérité faible (les en-têtes de sécurité majeurs — CSP, `X-Content-Type-Options`, options de cadrage — sont déjà positionnés). **Remédiation proposée.** Ajouter un en-tête `Permissions-Policy` restrictif dans l'intergiciel d'en-têtes de sécurité ; évaluer COOP/COEP selon les besoins d'isolation d'origine. **Résolution (v1.1).** En-tête `Permissions-Policy` restrictif ajouté (seuls `clipboard-write` et `fullscreen` restent autorisés pour l'origine) ainsi que `Cross-Origin-Opener-Policy: same-origin`. COEP n'est pas positionné : aucune fonctionnalité ne requiert l'isolation inter-origines.

D10 — Image de build front-end `node:20-slim`

Contexte. L'étape de compilation du frontend du `Dockerfile` repose sur `node:20-slim`, et le job `pnpm-audit` du workflow `security` utilise également Node 20. Node 20 n'est plus maintenu depuis avril 2026 (point ouvert N2 du rapport d'audit, document 14). **Risque.** Faible : Node n'intervient qu'à la construction et n'est pas présent dans l'image finale (distroless). L'absence de correctifs de sécurité de l'outil de build reste toutefois un écart de chaîne d'approvisionnement. **Remédiation proposée.** Migrer l'image de build et la CI vers une version LTS maintenue de Node, puis vérifier build, lint et tests frontend.

D11 — Images Docker non épinglées par empreinte

Contexte. Les images de base (`node:20-slim`, `rust:slim-bookworm`, `gcr.io/distroless/static-debian12:nonroot`) sont référencées par étiquette, sans empreinte `@sha256:` (point ouvert N3). Les actions CI, elles, sont déjà épinglées par empreinte de commit. **Risque.** Construction non reproductible ; une étiquette déplacée ou compromise en amont serait intégrée sans contrôle. **Remédiation proposée.** Épingler chaque image par empreinte (en conservant l'étiquette en commentaire) ; l'écosystème `docker` de Dependabot proposera alors les mises à jour d'empreinte.

D12 — Chaîne d'outils Rust non figée en CI

Contexte. Les workflows installent la chaîne `stable` (et `nightly` pour le fuzzing) sans version fixée ; aucun `rust-toolchain.toml` n'existe (point ouvert N4). **Risque.** Une nouvelle version de `clippy` (exécuté avec `-D warnings`) peut faire échouer la CI sans modification du code, comme constaté en octobre 2026 avec clippy 1.99 (nouveau lint sur les tris). **Remédiation proposée.** Ajouter un `rust-toolchain.toml` fixant la version stable (et une version datée de nightly pour le fuzzing), mise à jour volontairement.

D13 — Montées de version majeures en attente

Contexte. Dependabot (configuré depuis octobre 2026) a ouvert 22 pull requests, dont des montées majeures non fusionnables en l'état : axum 0.8 (actuellement 0.7), React 19 (actuellement 18), Tailwind CSS 4 (actuellement 3), TypeScript 6 (actuellement 5), rusqlite (actuellement 0.32), totp-rs 6 (actuellement 5) et l'image Node 26. **Risque.** Écart croissant avec les versions maintenues ; correctifs de sécurité futurs potentiellement réservés aux nouvelles versions majeures ; coût de migration qui augmente avec le temps. **Remédiation proposée.** Instruire chaque montée majeure comme un chantier dédié (branche, adaptation du code, tests complets), par ordre de risque : dépendances serveur exposées (axum, totp-rs, rusqlite) d'abord,

puis outillage frontend. Les mises à jour mineures et correctives groupées sont fusionnées au fil de l'eau (runbook R11, document 10).

D14 — Règles ESLint d'hygiène React en avertissement

Contexte. L'analyse ESLint (`pnpm lint`) est bloquante en CI pour les règles de sécurité (puits XSS, expressions régulières vulnérables au ReDoS). Quatre règles d'hygiène React restent en avertissement pour ne pas bloquer le pipeline sur l'existant : `@eslint-react/no-array-index-key` , `@eslint-react/set-state-in-effect` , `@eslint-react/static-components` et `@eslint-react/exhaustive-deps` . **Risque.** Défauts latents de rendu ou d'état (dépendances d'effet incomplètes, clés instables) ; les avertissements tendent à s'accumuler s'ils ne bloquent rien. **Remédiation proposée.** Corriger les occurrences existantes, puis passer ces règles en erreur.

D15 — Vérifications de sécurité hors CI

Contexte. Les constats de l'audit d'octobre 2026 sont couverts par des tests unitaires (103 tests backend, 114 tests frontend), mais la vérification de bout en bout de H2, M1, M2 et M4 repose sur des scripts locaux non versionnés, et celle de F1 à F5 sur des contrôles manuels sur une instance locale. Il n'existe pas encore de tests d'intégration HTTP du backend. **Risque.** Régression de contrôle d'accès non détectée par la CI : les scanners ne voient pas les failles de logique d'autorisation. **Remédiation proposée.** Mettre en œuvre la matrice d'autorisation et les tests de non-régression nommés prévus par le plan directeur des scans (document 15), sous forme de tests d'intégration exécutés à chaque pull request.

D16 — Verrouillage d'un lien public par un tiers (N1)

Contexte. Les tentatives de mot de passe sur `/s/<token>` et `/r/<token>` sont comptées sur deux clés : le lien et l'IP. La clé « lien » borne une attaque répartie sur de nombreuses IP, mais permet à quiconque connaît l'URL de verrouiller le lien 15 minutes pour tous ses destinataires en 5 échecs. **Risque.** Déni de service ciblé et renouvelable sur un lien ; pas d'impact sur la confidentialité. **Décision attendue.** Arbitrage produit entre protection contre la force brute et disponibilité (par ex. seuil plus élevé pour la clé « lien », ou verrouillage progressif). Contournement d'exploitation : runbook R9 (document 10).

D17 — Lien public créé par un destinataire de partage interne (N5)

Contexte. `POST /api/shares` n'exige que `validate_access(FileRead)` , qui tient compte des partages internes et des dossiers partagés : le destinataire d'un partage interne peut donc publier un lien public anonyme vers la ressource reçue. La création d'une demande de téléversement, elle, exige déjà l'accès propre (`validate_own_access`). **Risque.** Diffusion hors du cercle prévu par le propriétaire de la ressource. Le lien reste toutefois révoqué dès que son créateur perd l'accès en lecture (contrôle à chaque téléchargement). **Décision attendue.** Arbitrage produit ; l'alignement sur `validate_own_access` est l'option privilégiée.

D18 — Compteurs de verrouillage en mémoire

Contexte. Les cinq limiteurs (IP, utilisateur + IP, compte, 2FA, liens publics) sont des tables en mémoire du processus, purgées toutes les 30 minutes. **Risque.** Un redémarrage lève tous les verrous en cours, y compris ceux d'un attaquant ; les compteurs ne sont pas partagés entre instances, ce qui s'ajoute aux prérequis d'un éventuel déploiement multi-instances (voir D8 et document 10, §8.1). **Remédiation proposée.** Acceptable en déploiement mono-instance. À reconsidérer (persistance des verrous actifs en base, ou stockage partagé) si une architecture multi-instances est retenue.

---

6. Plan de résorption priorisé

Priorité 1 — court terme (fiabilité et assurance sécurité)

- **D1** — Rebasage des métadonnées au renommage/déplacement (cohérence fonctionnelle).
- **D5** — Scan DAST actif authentifié de l'API (couverture de sécurité).
- **D7** — Amorcer les tests frontend automatisés sur les modules critiques.

Priorité 2 — moyen terme (durcissement et cohérence)

- **D9** — En-têtes `Permissions-Policy` (gain rapide, effort faible).
- **D6** — Trancher la source unique de verrouillage des dépendances frontend.
- **D8** — Étude de scalabilité de l'accès base (pool de connexions).
- **D3** — Veille sur une compilation statique compatible PIE (risque accepté entre-temps).

Priorité 3 — opportuniste

- **D2** — Nettoyage des clés i18n orphelines.
- **D4** — Levée des exceptions `rsa` et `crypto-bigint` conditionnée à l'évolution des dépendances (risque accepté).

Mise à jour du 27 octobre 2026 (v1.3) — D1, D2, D6, D7 et D9 sont résolues. Ordre de traitement des entrées ouvertes :

- **P1** : **D15** (tests d'intégration d'autorisation en CI), **D17** (décision N5), **D5** (DAST authentifié).
- **P2** : **D10**, **D11**, **D12** (gains rapides de chaîne de build), **D16** (décision N1), **D13** (montées majeures, en commençant par les dépendances serveur), **D8**, **D3** (veille).
- **P3** : **D14** (règles React en erreur), **D18** (à revoir avec toute évolution multi-instances), **D4** (veille).

---

7. Références

- **Ward Cunningham**, « The WyCash Portfolio Management System », OOPSLA 1992 — origine du concept de dette technique.
- **Martin Fowler**, « Technical Debt Quadrant » — grille de qualification des dettes.
- **Document 10** — Dossier d'exploitation (DEX) (corpus documentaire FileFlow).
- **Document 11** — Observabilité (SLI/SLO) (corpus documentaire FileFlow).
- **Document 14** — Rapport d'audit de sécurité d'octobre 2026 (points ouverts N1 à N6).
- **Document 15** — Plan directeur des scans et rapports de sécurité.